

Turning Video Models into Generalist Robot Policies

Sizhe Lester Li^{1,*} Evan Kim^{1,*} Xingjian Bai^{1,*}
 Tong Zhao² Tao Pang² Max Simchowitz^{3,4} Vincent Sitzmann¹
¹MIT ²Independent Researcher ³CMU ⁴Amazon FAR



Figure 1: **Controlling robots across embodiments, skills, and environments.** We demonstrate that a 14B video generative model, paired with an inverse dynamics model carefully designed for action-following, can solve a wide variety of robotic tasks, ranging from zero-shot pick-and-place tasks on a real-world Panda arm to contact-rich re-orientation of a cube with a 16-DoF robotic hand.

Abstract: Video generative models have emerged as a promising robotics backbone, capable of generating videos that depict the completion of complex tasks across embodiments and environments. Recent work proposes robot foundation models that jointly predict future observations and actions by finetuning video models with action-labeled data. In this paper, we test the limits of an alternative approach: leave the video planner as-is while training an embodiment-specific inverse dynamics model (IDM). This decoupling offers several natural benefits: the video planner remains embodiment-agnostic, different video models can be interchanged easily without re-training the IDM, and the IDM can be independently trained with readily available self-play data. We present a closed-loop, video-to-action policy that combines an action-free video world model with a carefully-designed IDM based on the robot embodiment Jacobian. We demonstrate that our IDM design is both data-efficient and scalable to high-dimensional action spaces. Our policy, which we coin the Video-to-Embodied Robot Action Model (VERA), achieves strong performance across simulated and real-world benchmarks, including zero-shot Panda arm manipulation and 16-DoF Allegro-hand dexterous cube re-orientation. The same video planner can be used across multiple embodiments by pairing it with different embodiment-specific IDMs. Our results show that decoupled video planning plus faithful video-to-action translation is a viable alternative route towards zero-shot, cross-embodiment, and generalizable robot control. More results are available on our project website ².

Keywords: Video Generative Models, Inverse Dynamics, Generalist Robot Policies

*Equal contribution.

²Project Website: <https://vera.csail.mit.edu>

1 Introduction

The core challenge of robot intelligence is one of generalization: we seek a system that can control a wide variety of embodiments to solve unseen tasks in new environments.

In natural language processing, the generality of large language models (LLMs) and vision-language models (VLMs) emerged through jointly scaling models and datasets. Vision-language-action (VLA) models extend pretrained vision-language backbones with action outputs in an attempt to transfer their generality to robotics tasks [1–5]. However, different from the success of fine-tuning LLMs to VLMs, where web-scale, paired text and image data was available, no web-scale robot action data exists. In practice, it has proven challenging to achieve strong levels of generalization to new tasks, embodiments, and environments with VLAs [6]: robot actions appear to be too distinct to inherit the generality of text and image models through fine-tuning on comparatively small action datasets.

Video generative models have been proposed as an alternative robotics backbone. They can generate “visual plans”, imagining what it would look like for a robot to solve a specified task. Video models are not intrinsically tied to the action space of any one robot, and video data is abundant. However, we must now solve the problem of translating the video plan into robot actions. One route is to integrate actions into the video backbone itself, either through joint video-and-action prediction, known as *world-action models* (WAMs), or through action-conditioned world models paired with inference-time search [7–13]. This route has produced impressive results, but, as with VLAs, requires a particularly rare kind of data for training: paired video, action, and text data.

In this work, we test the limits of an alternative approach to leveraging video models for robotics: a separate *inverse dynamics model* (IDM) that takes video as input and outputs corresponding robot actions. This separation has several key advantages. First, an IDM can be trained with *task-agnostic* video-action data, which can be more readily generated e.g. via self-play. Second, the *same* generative model can be shared across multiple embodiments by swapping the IDM. Lastly, stronger video backbones can be swapped in without retraining the IDM. However, the performance of the end-to-end pipeline is now critically dependent on the performance of the IDM: An inaccurate IDM leads to failures even if the video plan was perfect. Prior IDMs regress actions directly from a pair of generated frames with an unconstrained network [14–16]. We find that, with limited action data, IDMs often generalize poorly out of distribution. Moreover, their accuracy degrades substantially as the complexity of the action space grows with more complex embodiments (Sec. 4).

To overcome these challenges, we introduce the *Jacobian-IDM* (J-IDM), which predicts actions through the inversion of a learned tangent map between action perturbations and pixel motion. We show that such a structure is both data-efficient and scales gracefully with embodiment complexity. We then pair J-IDM with a 14B video model adapted from the Large Video Planner [17]: the video model predicts a visual lookahead, J-IDM translates this lookahead into an executable action chunk, the system executes and observes, and finally replans based on the rollout. We call our closed-loop video-to-action policy Video-to-Embodied Robot Action Model (VERA).

VERA is successful in simulated and real settings. On a real Panda arm, VERA performs zero-shot language-conditioned manipulation in a new physical setup with varied camera placements, scene configurations, and prompts requiring visual and linguistic reasoning. On a real Allegro hand [18], VERA performs RGB-only 16-DoF in-hand cube reorientation. In simulation, J-IDM outperforms prior IDM baselines. Furthermore, we validate that a video planner trained across our real-world embodiments can be used with different embodiment-specific J-IDMs. Lastly, we situate VERA against recent VLA and WAM baselines on real-world manipulation tasks. Our results show that video planning decoupled from a faithful inverse dynamics model is a promising and modular alternative for zero-shot, multi-embodiment, and generalizable robot control.

2 Related Work and Preliminaries

Behavior Cloning and Vision-Language-Action Models. Behavior cloning (BC) remains the dominant recipe for robot manipulation, learning actions directly from observations and task context,

from offline visuomotor BC and language-conditioned imitation to more expressive sequence models based on action chunking, diffusion, and 3D action-centric representations [19–24]. Modern VLA and generalist robot policies largely scale this same action-supervised interface: RT-1 and RT-2 attach robot actions to large visual/language backbones, while PaLM-E, RoboCat, Open X-Embodiment / RT-X, Octo, OpenVLA, and the π -family extend this paradigm to broader data mixtures, embodiments, and action spaces [1–5, 25–28]. Despite their scale and generality across *task* semantics, these methods still couple perception and control within a single embodiment-specific model. VERA instead keeps the foundation model in video space and learns separate embodiment-specific video-to-action bridges for each embodiment.

Video Generative Models. Large-scale video pretraining has produced increasingly capable generative models with emergent visual priors and rudimentary physical reasoning [29, 30]. We build on the WAN family of open-weight video diffusion transformers [31] and train with Diffusion Forcing [32, 33], which enables variable-context conditioning.

Joint Video-and-Action Models. A complementary line of work trains a single backbone to predict both future visual observations and robot actions, using video prediction as dense dynamics supervision while retaining an action head for execution. GR-2 jointly models video, language, and actions for manipulation [7]; Video Prediction Policy learns predictive visual representations for policy learning [8]; and LingBot-VA and DreamZero / World Action Models co-predict world states and actions with a video model backbone [9, 10]. Related action-conditioned world models and simulators use generated futures for planning or search [11, 12, 34]. These approaches couple visual forecasting and action generation, whereas VERA separates them: the video model proposes an action-free visual plan, and an embodiment-specific IDM translates visual motion into controls.

Video Models with Inverse Dynamics Models. Most closely related to VERA are methods that decouple robot control into a pixel-space planner and a translator [14, 15, 17, 35, 36]. A similar factorization has also begun to appear in industrial systems: concurrent with this work, Rhoda AI’s Direct Video-Action system achieves closed-loop, long-horizon bimanual manipulation in real-world settings [16]. This result provides encouraging evidence that decoupled architectures can scale to real deployments.

3 The Video-to-Embodied Robot Action Model

In this section, we first articulate the central technical challenge of using a video generative model as a robot policy — translating predicted video into executable actions — and identify the desired traits of a good translator (Sec. 3.1). We then introduce our proposed solution, the Jacobian IDM (Sec. 3.2) and describe how we put it all together in a closed-loop with a video model in Sec. 3.3.

3.1 The Video-to-Action Problem

A video world model π_{vid} produces a visual plan in pixel space: given an observation history $\mathbf{o}_{\leq t}$ and a goal g , it samples

$$(\hat{\mathbf{o}}_{t+1}, \hat{\mathbf{o}}_{t+2}, \dots, \hat{\mathbf{o}}_{t+M}) \sim \pi_{\text{vid}}(\cdot \mid \mathbf{o}_{\leq t}, g). \quad (1)$$

Robots, however, do not act in pixels; they execute embodiment-specific commands such as position, velocity, or torque commands. The central question for the Video Model-IDM direction is therefore: *how do we recover actions $\hat{\mathbf{a}}_{t:t+M-1}$ from video frames?*

Desiderata: Faithfulness, data-efficiency, and scaling with DoFs. A good IDM should, first and foremost, be *faithful*: its inferred actions should reliably reproduce the predicted visual transition when executed. Beyond this, because action-labeled data is typically limited, it should be *data-efficient*, best if it only requires self-play data to train. Finally, it should *scale with action complexity*, allowing it to work for a variety of complex embodiments.

IDMs require careful designs. Current IDMs come in two flavors. The first is hand-crafted methods based on retargeting or 3D representations [15, 17]. The second is direct parameterization via a neural network that learns to regress $\hat{\mathbf{a}}_t$ from the image pair $(\hat{\mathbf{o}}_t, \hat{\mathbf{o}}_{t+1})$ end-to-end [14]. Such an approach,

which we call a *Direct IDM* (D-IDM), has value in its simplicity. However, under limited data and complex embodiments, a more structured IDM may be needed to satisfy the previous criteria. In Section 4 we show empirically that unstructured D-IDMs may sacrifice *faithfulness* under (i) data constraints and (ii) increasing action dimension.

3.2 The Jacobian Inverse Dynamics Model

The embodiment Jacobian. The problem of relating infinitesimal actions to motion is solved in classical robotics by the embodiment Jacobian. Let $\mathbf{a} \in \mathbb{R}^n$ denote the robot’s action vector and $\mathbf{x}_i \in \mathbb{R}^3$ the 3D location of body point i . At a given state—observed as $\mathbf{o}$ —the embodiment Jacobian

$$\mathbf{J}_i(\mathbf{o}) = \left. \frac{\partial \mathbf{x}_i}{\partial \mathbf{a}} \right|_{\mathbf{o}} \in \mathbb{R}^{3 \times n} \quad (2)$$

(Eq. 2) is the local linear map from action perturbations to 3D point motion: $\delta \mathbf{x}_i \approx \mathbf{J}_i(\mathbf{o}) \delta \mathbf{a}$. However, 3D points are never observed directly, making such a Jacobian difficult to learn directly.

Image-space Jacobian field. This motivates predicting the Jacobian directly in image space, with $\mathbf{o}$ itself as the conditioning variable. Given a single image $\mathbf{o} \in \mathbb{R}^{H \times W \times 3}$, an image-conditioned transformer $\mathbf{J}_\theta$ outputs a dense field

$$\mathbf{J}_\theta(\cdot, \mathbf{o}) : \{1, \dots, H\} \times \{1, \dots, W\} \longrightarrow \mathbb{R}^{2 \times n}, \quad (3)$$

which assigns to every pixel $\mathbf{p}$ a $2 \times n$ matrix that linearizes how an action increment $\delta \mathbf{a} \in \mathbb{R}^n$ moves that pixel:

$$\delta \mathbf{p} = \mathbf{J}_\theta(\mathbf{p}, \mathbf{o}) \delta \mathbf{a}, \quad (4)$$

where $\delta \mathbf{p}$ is the per-pixel measurement of the observation change $\delta \mathbf{o}$, obtained in practice from an off-the-shelf optical-flow estimator (Eq. 5). Prior work parameterized this Jacobian as a 3D field, lifting per-pixel motion to 3D via volume rendering of a NeRF-style scene representation [37]; we drop the 3D scene representation, which lets us scale $\mathbf{J}_\theta$ as a single, large image-conditioned transformer. At test time, we recover the action by inverting Eq. 4.

Joint forward-inverse training objective. We train $\mathbf{J}_\theta$ on a dataset of $(\mathbf{o}_t, \delta \mathbf{a}_t, \mathbf{o}_{t+1})$ tuples. For each tuple, we extract a dense optical flow field $\mathbf{v}_t \in \mathbb{R}^{H \times W \times 2}$ between $\mathbf{o}_t$ and $\mathbf{o}_{t+1}$ using off-the-shelf motion estimators [38–41]. We supervise $\mathbf{J}_\theta$ with a joint forward-inverse loss combining the per-pixel Charbonnier objective $\rho(x) = \sqrt{x^2 + \varepsilon^2}$ on the predicted pixel motion with an action-reconstruction term using a λ -regularized pseudoinverse $\mathbf{J}_\theta^{\dagger, \lambda}$:

$$\mathcal{L} = \underbrace{\sum_{\mathbf{p}} \rho(\mathbf{J}_\theta(\mathbf{p}, \mathbf{o}_t) \delta \mathbf{a}_t - \mathbf{v}_t(\mathbf{p}))}_{\text{forward (pixel-flow)}} + w_a \underbrace{\sum_{\mathbf{p}} \|\delta \mathbf{a}_t - \mathbf{J}_\theta^{\dagger, \lambda}(\mathbf{p}, \mathbf{o}_t) \mathbf{v}_t(\mathbf{p})\|_2^2}_{\text{inverse (action reconstruction)}}, \quad (5)$$

with $w_a = 0.3$; see App. B for training details.

From forward model to IDM. At inference, we are given two consecutive predicted frames $(\hat{\mathbf{o}}_t, \hat{\mathbf{o}}_{t+1})$ from the video planner. We extract their optical flow $\mathbf{v}_t$ with the same off-the-shelf motion estimator used in training and recover the action via the λ -regularized pseudoinverse:

$$\widehat{\delta \mathbf{a}}_t = \mathbf{J}_\theta^{\dagger, \lambda}(\hat{\mathbf{o}}_t) \mathbf{v}_t \triangleq \arg \min_{\delta \mathbf{a} \in \mathbb{R}^n} \sum_{\mathbf{p}} \|\mathbf{J}_\theta(\mathbf{p}, \hat{\mathbf{o}}_t) \delta \mathbf{a} - \mathbf{v}_t(\mathbf{p})\|_2^2 + \lambda \|\delta \mathbf{a}\|_2^2. \quad (6)$$

3.3 Video World Model as a Closed-Loop Robot Planner

VERA uses a video world model as a planner in observation space, as illustrated in Fig. 2. Given some observations and conditioning signal (typically text), it predicts how the scene should visually evolve. Two design choices distinguish our planner from a vanilla video model.

Action-free robot post-training. We instantiate the planner from a pretrained video model [17, 31] and lightly adapt it to robot video-only data [42]; the post-training objective is generative video prediction with *no action head*. Two modifications make it usable as a policy: video-to-video finetuning (N context frames in, M future frames out) for autoregressive rollouts, and a multi-view variant that tiles cameras into a single canvas. Architectural details are in App. A.1–A.2.

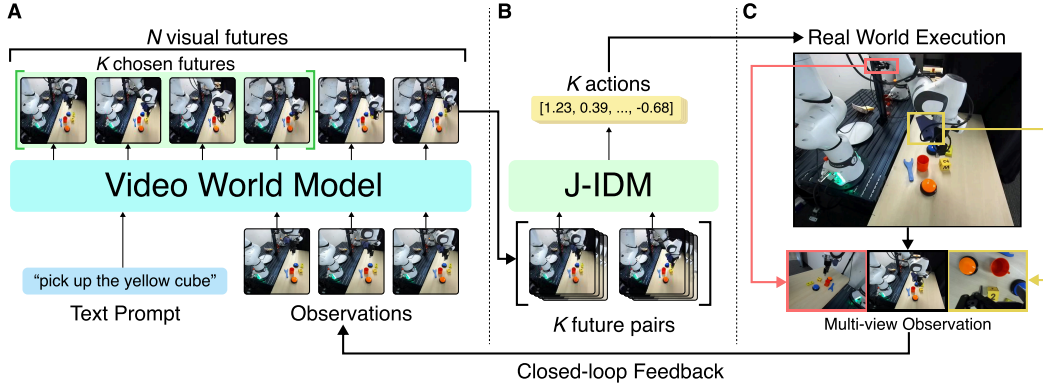


Figure 2: **Translating video to actions.** **a**, Given context frames, our video world model rolls out a short visual plan. **b**, The Jacobian IDM inverts each step of this path into a chunk of low-level actions. **c**, The chunk is executed and observations return to the video model for closed-loop execution.

Executing on a visual chunk. Although the planner generates M future frames, the controller commits to executing only the first K predicted frames. The Jacobian IDM converts this committed prefix into a chunk of robot actions by applying Eq. 6 independently to every pair of adjacent frames, using the current observation $\mathbf{o}_t$ as the anchor:

$$\hat{\mathbf{a}}_{t:t+K} = G_\phi(\mathbf{o}_t, \hat{\mathbf{o}}_{t+1}, \dots, \hat{\mathbf{o}}_{t+K}),$$

yielding a chunk length of K actions. This allows the planner to reason over a longer visual look-ahead while the controller stays grounded through frequent feedback. Consistent with prior findings on action chunking, we observe that this chunking improves performance [23, 43]; we ablate the choice of K in Fig. 8.

Closed-loop replanning. After executing the chunk, the robot appends the newly observed frames to its history and queries the planner again. The execution horizon K governs the feedback–smoothness trade-off: larger values produce longer locally-coherent chunks and amortize video generation time; smaller values reduce drift during execution. The full procedure is given in Alg. 1.

4 Experiments

We evaluate the VERA system in closed-loop manipulation, isolate the Jacobian IDM against alternative IDM variants, and compare to state-of-the-art VLA and world-action baselines [5, 10].

Experiment Setup. Unless otherwise noted, VERA uses a Wan-family pretrained video model [31] as the planner and an embodiment-specific J-IDM initialized from VGGT [44] as the translator. We evaluate in simulation (PushT, MimicGen Panda [45], Allegro-Sim cube reorientation [46]) and on hardware (Panda-Real post-trained on DROID [42], Allegro-Real dexterous reorientation), reporting closed-loop success and task progress; full setup and video-model ablations are in Appendix A–A.2.

4.1 Main Results

Jacobian IDMs enable faithful video-to-action translation. We found that our Jacobian IDM can faithfully translate video chunks predicted by a world model into action chunks executed on the robot. Qualitatively, across environments, we have executions that are consistent with our generations, as shown in Fig. 3,4. Faithful translation naturally provides the robotic system with the capabilities of video models: visuospatial reasoning, prompt-following, embodiment generalization, and task generalization. These will be discussed throughout the rest of this section.

VERA performs zero-shot, prompt-following manipulation tasks on a Panda arm. Just as any video model can generate videos zero-shot in new environments, when armed with a translator, VERA performs zero-shot manipulation. Throughout our evaluations, we varied lighting conditions, camera configurations, and object orientations and found it robust to such changes. Our robot can perform manipulations that follow complex prompts, as demonstrated in Fig. 1, and perform complex vision-language reasoning tasks, as will be expanded upon in Sec. 4.3.

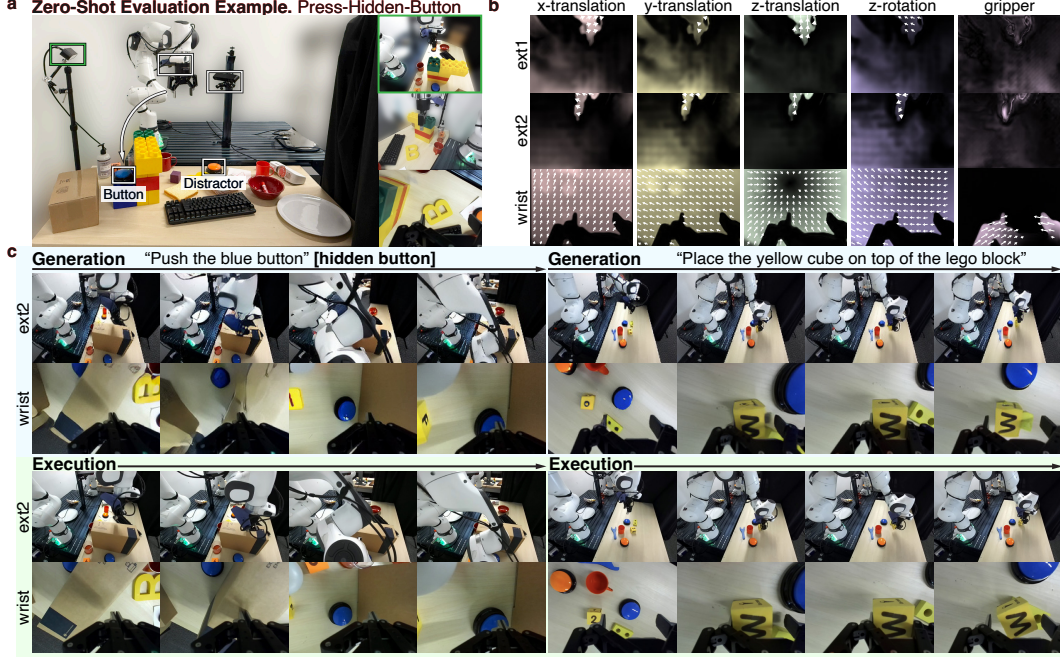


Figure 3: **Zero-Shot Evaluation on Panda Arm.** **a**, After training on DROID [42], we deploy zero-shot on a Panda in an unseen scene with ad-hoc camera placements; in *Press-Hidden-Button*, the blue target is visible from only one of three views, with an orange distractor probing instruction grounding. **b**, Visualization of the predicted Jacobian: each column of $\mathbf{J}_\theta$ is assigned a fixed RGB color and rendered, per pixel, weighted by the norm of the corresponding Jacobian column. Columns: action channels; rows: viewpoints. **c**, Generated frames and executions shown side-by-side with the text prompt; actions remain closely aligned with the generated frames.

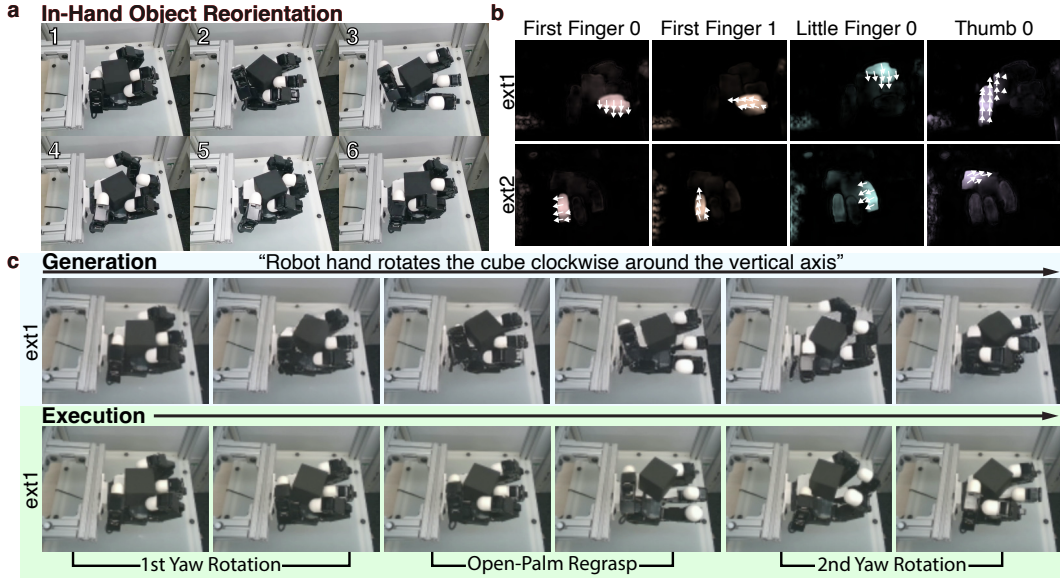


Figure 4: **In-Hand Object Reorientation.** **a**, Cube-reorientation demonstrations [46] with three language instructions (clockwise, counter-clockwise, random); we evaluate prompt-conditioned in-hand reorientation. **b**, Predicted Jacobian visualized as in Fig. 3b (columns: selected action channels out of 16 total; rows: viewpoints). **c**, Generated frames and executions side-by-side with prompt; the model orchestrates dexterous finger motions.

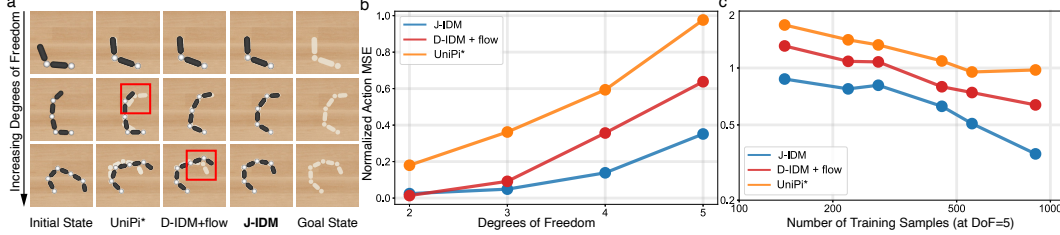


Figure 5: **J-IDM scales with action dimensionality.** **a**, Rollouts of recovered actions for direct-IDM baselines and J-IDM as DoFs grow. **b**, At a fixed data budget, the J-IDM gap over direct-IDMs widens with DoF. **c**, At high DoF, J-IDM has a better data-accuracy trade-off across training sizes.

Table 1: **Closed-loop results in simulated environments.** Each cell reports success rate / task progress (%), $\uparrow$). J-IDM yields higher closed-loop success than a UniPi*-style direct inverse-dynamics baseline [14] across planar pushing, 7-DoF arm manipulation, and 16-DoF dexterous manipulation.

Model	Allegro-Sim	Panda-Sim (MimicGen)	PushT-Sim
Uni-Pi*	0.0 $\pm$ 0.0 / 0.0 $\pm$ 0.0	0.0 $\pm$ 0.0 / 0.0 $\pm$ 0.0	74.4 $\pm$ 3.4 / 84.8 $\pm$ 2.8
J-IDM (ours)	70.0 $\pm$ 14.5 / 70.0 $\pm$ 14.5	94.0 $\pm$ 3.4 / 94.0 $\pm$ 3.4	92.5 $\pm$ 2.1 / 95.5 $\pm$ 1.6

VERA admits multi-embodiment. Our video model which is finetuned across DROID and sim and real allegro hands — two vastly different embodiments — is able to generate successful video plans on both. Then, armed with two embodiment-specific J-IDMs, is able to control both a panda arm and the contact-rich re-orientation of a block via its fingers. As seen in Fig. 4, the latter is a multi-stage and challenging task for a purely visual system. However, armed with a strong video model, a sufficient translator, and closed loop control, VERA is able to complete the task.

4.2 Designing a Faithful IDM for Video-to-action Translation

The closed-loop results above rely on *faithful* video-to-action translation. We now isolate this property and ask what makes it possible. By comparing a Jacobian-parameterized IDM against unstructured D-IDM baselines under controlled data and DoF budgets, we find the gain comes specifically from the Jacobian-based constraint. The J-IDM is one concrete instantiation of that constraint; the rest of this subsection compares it against unstructured D-IDM baselines.

Experiment Setup. We compare our J-IDM against blackbox D-IDM approaches. In particular, we test against two models: one which takes in a pair of images (i.e. $\mathbf{o}_t$ and $\mathbf{o}_{t+1}$) and another which takes in an image and the optical flow between the two images. All models are kept at the same parameter count, trained on the same data, and share the same architecture with the exception of the decoding head. The flow-conditioned model serves as an ablation of our approach, since it has the same input-output behavior, but lacks our structured representation. The image-pair model is our best-effort replication of UniPi [14]. Because we use a different architecture to ensure a fair comparison, we refer to this baseline as UniPi*. More details can be found in Appendix B.3.

J-IDMs are performant under data and complexity constraints. A controlled 2D “toy finger” study (Fig. 5) sweeps degrees of freedom and self-play data quantity. The structured parameterization brings two benefits over an unstructured regressor: (1) at fixed data, J-IDM is the only model preserving faithful reconstruction as DoFs grow (Fig. 5a,b); and (2) at fixed DoFs (=5), it is approximately $2\times$ more data-efficient (Fig. 5c).

Faithful translation results in higher success rates. This fidelity carries downstream. In Tab. 2, J-IDM achieves the lowest action-reconstruction MSE on all but one of the tested environments. Improved reconstruction translates to task success by preserving the video model’s “dreams”: with the planner held fixed, J-IDM outperforms UniPi* on every closed-loop task (Tab. 1).

Table 2: **Action reconstruction for video-to-action translation.** Reconstruction MSE on held-out visual transitions. Jacobian IDM achieves the lowest MSE on Allegro-Sim, PushT-Sim, and 5-joint fingers, and remains competitive on Panda-Sim. **Best / second-best.**

Model	Action reconstruction MSE ↓			
	Allegro-Sim	Panda-Sim (MimicGen)	PushT-Sim	5-joint fingers
Uni-Pi*	0.063	0.38	0.071	0.047
D-IDM + Flow	<u>0.044</u>	0.09	<u>0.059</u>	<u>0.030</u>
J-IDM (ours)	0.031	<u>0.19</u>	0.046	0.017

4.3 VERA and other Robotic Foundation Models

We compare VERA against state-of-the-art VLA and WAM baselines ($\pi_{0.5}$, DreamZero) on an in-house Panda manipulation suite, using DROID-trained checkpoints across all systems (Fig. 6).

Instruction following on basic tasks. On basic “push A”/“pick up B” tasks, DreamZero attains 90%, VERA 60%, and $\pi_{0.5}$ 30%. VERA’s failures generally arise from the video-to-action step: VERA’s dreamed futures on failing rollouts consistently complete the task, but the translation to action lacks fidelity. $\pi_{0.5}$, on the other hand, frequently disregards the instruction and acts on the wrong object.

Instruction following on challenging tasks. We further design two sets of harder, reasoning-heavy prompts: *location-based* prompts (e.g., “push the button on top of the paper”) that do not directly reveal the target, and *semantic-based* prompts (e.g., “push the button matching the wrench’s color”) that require visual grounding. While video-to-action translation is the bottleneck for VERA on basic tasks, the strength of the underlying video model allows VERA to remain performant: DreamZero and $\pi_{0.5}$ frequently act on incorrect objects entirely, whereas VERA succeeds, suggesting that decoupling the planner from the IDM preserves more of the video model’s reasoning.

Reasoning through occlusions. To further test multi-view reasoning, we design the “find the button” challenge: a button is hidden behind a wall, visible from only one of three cameras, amid distractor props on a cluttered table (Figs. 3, 7). DreamZero and $\pi_{0.5}$ struggle to locate and navigate around the wall. VERA, on the other hand, produces coherent plans that the J-IDM is able to execute. Moreover, for DreamZero, the dreamed future itself fails to produce a plan that finds and presses the button, indicating that the failure originates in the video branch. This is consistent with the hypothesis that preserving the video branch on its own allows the model to retain video-model reasoning capabilities that an end-to-end head would otherwise dilute.

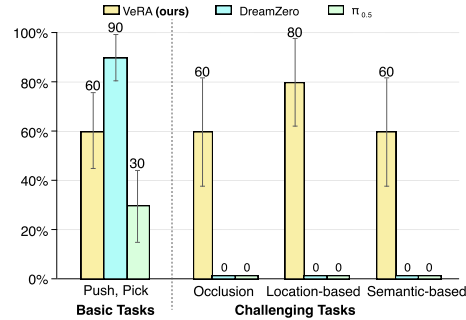


Figure 6: **Real-world comparison with robot foundation model baselines.** VERA remains competitive on basic tasks and substantially better on challenging tasks.

5 Discussion and Conclusion

Conclusion. We have shown that strong video models, paired with a faithful IDM, can control diverse embodiments and solve diverse tasks. Our Jacobian IDM is one such instantiation: data-efficient, scaling with action dimensionality, and preserving the visual reasoning of the planner that an end-to-end action head would otherwise dilute.

Limitations. VERA still requires robot-specific video for planner post-training, depends on off-the-shelf optical-flow trackers [38, 40] for J-IDM supervision, and cannot reason about force-based control [47] from RGB alone. Faithful action recovery also degrades when the predicted transition has little observable pixel motion.

Future Work. Promising directions include sim-to-real J-IDM transfer to remove the need for real-robot action data, planner co-training across humans and robots, and an embodiment-conditioned Jacobian that serves multiple morphologies—motivating careful IDM design as video world models improve.

Acknowledgments

This work was supported by the National Science Foundation under Grant No. 2211259, by the Intelligence Advanced Research Projects Activity (IARPA) via Department of Interior/ Interior Business Center (DOI/IBC) under 140D0423C0075, by the Amazon Science Hub, by the MIT-Google Program for Computing Innovation, by Advanced Micro Devices, Inc. under the AMD University Program’s support of the MIT Hardware Consortium, and by a 2025 MIT Office of Research Computing and Data Seed Grant.

References

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, G. Lam, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- [3] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. URL <https://octo-models.github.io/>.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. doi:10.15607/RSS.2025.XXI.010. URL <https://www.roboticsproceedings.org/rss21/p010.html>.
- [5] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, b. ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In J. Lim, S. Song, and H.-W. Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 27–30 Sep 2025. URL <https://proceedings.mlr.press/v305/black25a.html>.
- [6] J. Wang, M. Leonard, K. Daniilidis, D. Jayaraman, and E. Hu. Evaluating pi0 in the wild: Strengths, problems, and the future of generalist robot policies, 2025.
- [7] C.-L. Cheang, G. Chen, Y. Jing, T. Kong, H. Li, Y. Li, Y. Liu, H. Wu, J. Xu, Y. Yang, H. Zhang, and M. Zhu. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024. URL <https://arxiv.org/abs/2410.06158>.
- [8] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/hu25g.html>. Spotlight.

- [9] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, Y. Shen, and Y. Xu. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026. URL <https://arxiv.org/abs/2601.21998>. Introduces LingBot-VA, a video–action co-prediction framework.
- [10] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, A. Malik, K. Lee, W. Liang, N. Ranawaka, J. Gu, Y. Xu, G. Wang, F. Hu, A. Narayan, J. Bjorck, J. Wang, G. Kim, D. Niu, R. Zheng, Y. Xie, J. Wu, Q. Wang, R. Julian, D. Xu, Y. Du, Y. Chebotar, S. Reed, J. Kautz, Y. Zhu, L. J. Fan, and J. Jang. World action models are zero-shot policies, 2026. URL <https://arxiv.org/abs/2602.15922>.
- [11] S. Yang, Y. Du, S. K. S. Ghasemipour, J. Tompson, L. P. Kaelbling, D. Schuurmans, and P. Abbeel. Learning interactive real-world simulators. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=sFyTZEqmUY>.
- [12] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/zhou25t.html>.
- [13] S. Li, Y. Gao, D. Sadigh, and S. Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- [14] Y. Du, M. Yang, B. Dai, H. Dai, O. Nachum, J. B. Tenenbaum, D. Schuurmans, and P. Abbeel. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1d5b9233ad716a43be5c0d3023cb82d0-Abstract-Conference.html.
- [15] P.-C. Ko, J. Mao, Y. Du, S.-H. Sun, and J. B. Tenenbaum. Learning to act from actionless videos through dense correspondences. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=Mhb5fpA1T0>.
- [16] Rhoda AI Research. Causal video models are data-efficient robot policy learners. Rhoda AI Research blog, 2026. URL <https://www.rhoda.ai/research/direct-video-action>.
- [17] B. Chen, T. Zhang, H. Geng, K. Song, C. Zhang, P. Li, W. T. Freeman, J. Malik, P. Abbeel, R. Tedrake, V. Sitzmann, and Y. Du. Large video planner enables generalizable robot control. *arXiv preprint arXiv:2512.15840*, 2025. URL <https://arxiv.org/abs/2512.15840>.
- [18] Wonik Robotics. Allegro hand. <https://www.allegrohand.com/sub/product/p.php?idx=1>, 2024. Accessed: 2026-05-07.
- [19] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 1678–1690. PMLR, 2022. URL <https://proceedings.mlr.press/v164/mandlekar22a.html>.
- [20] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn. BC-Z: Zero-shot task generalization with robotic imitation learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 991–1002. PMLR, 2022. URL <https://proceedings.mlr.press/v164/jang22a.html>.
- [21] M. Shridhar, L. Manuelli, and D. Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In K. Liu, D. Kulic, and J. Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 785–799. PMLR, 14–18 Dec 2023. URL <https://proceedings.mlr.press/v205/shridhar23a.html>.

- [22] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan. VIMA: Robot manipulation with multimodal prompts. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 14975–15022. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/jiang23b.html>.
- [23] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. doi:10.15607/RSS.2023.XIX.016. URL <https://www.roboticsproceedings.org/rss19/p016.html>. Action Chunking with Transformers (ACT).
- [24] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10–11):1684–1704, 2025. doi:10.1177/02783649241273668. URL <https://journals.sagepub.com/doi/10.1177/02783649241273668>.
- [25] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *arXiv preprint arXiv:2212.06817*, 2022.
- [26] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. Palm-e: An embodied multimodal language model. In *arXiv preprint arXiv:2303.03378*, 2023.
- [27] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, A. Laurens, C. Fantacci, V. Dalibard, M. Zambelli, M. Martins, R. Pevceviciute, M. Blokzijl, M. Denil, N. Batchelor, T. Lampe, E. Parisotto, K. Żoźna, S. Reed, S. G. Colmenarejo, J. Scholz, A. Abdolmaleki, O. Groth, J.-B. Regli, O. Sushkov, T. Rothörl, J. E. Chen, Y. Aytar, D. Barker, J. Ortiz, M. Riedmiller, J. T. Springenberg, R. Hadsell, F. Nori, and N. Heess. Robocat: A self-improving generalist agent for robotic manipulation, 2023. URL <https://arxiv.org/abs/2306.11706>.
- [28] O. X.-E. Collaboration, A. O’Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, A. Tung, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Gupta, A. Wang, A. Kolobov, A. Singh, A. Garg, A. Kembhavi, A. Xie, A. Brohan, A. Raffin, A. Sharma, A. Yavary, A. Jain, A. Balakrishna, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Wulfe, B. Ichter, C. Lu, C. Xu, C. Le, C. Finn, C. Wang, C. Xu, C. Chi, C. Huang, C. Chan, C. Agia, C. Pan, C. Fu, C. Devin, D. Xu, D. Morton, D. Driess, D. Chen, D. Pathak, D. Shah, D. Büchler, D. Jayaraman, D. Kalashnikov, D. Sadigh, E. Johns, E. Foster, F. Liu, F. Ceola, F. Xia, F. Zhao, F. V. Frujeri, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Feng, G. Schiavi, G. Berseth, G. Kahn, G. Yang, G. Wang, H. Su, H.-S. Fang, H. Shi, H. Bao, H. B. Amor, H. I. Christensen, H. Furuta, H. Bharadhwaj, H. Walke, H. Fang, H. Ha, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Abou-Chakra, J. Kim, J. Drake, J. Peters, J. Schneider, J. Hsu, J. Vakil, J. Bohg, J. Bingham, J. Wu, J. Gao, J. Hu, J. Wu, J. Wu, J. Sun, J. Luo, J. Gu, J. Tan, J. Oh, J. Wu, J. Lu, J. Yang, J. Malik, J. Silvério, J. Hejna, J. Booher, J. Tompson, J. Yang, J. Salvador, J. J. Lim, J. Han, K. Wang, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Black, K. Lin, K. Zhang, K. Ehsani, K. Lekkala, K. Ellis, K. Rana, K. Srinivasan, K. Fang, K. P. Singh, K.-H. Zeng, K. Hatch, K. Hsu, L. Itti, L. Y. Chen, L. Pinto, L. Fei-Fei, L. Tan, L. J. Fan, L. Ott, L. Lee, L. Weihs, M. Chen, M. Lepert, M. Memmel, M. Tomizuka, M. Itkina,

- M. G. Castro, M. Spero, M. Du, M. Ahn, M. C. Yip, M. Zhang, M. Ding, M. Heo, M. K. Srirama, M. Sharma, M. J. Kim, M. Z. Irshad, N. Kanazawa, N. Hansen, N. Heess, N. J. Joshi, N. Suenderhauf, N. Liu, N. D. Palo, N. M. M. Shafiullah, O. Mees, O. Kroemer, O. Bastani, P. R. Sanketi, P. T. Miller, P. Yin, P. Wohlhart, P. Xu, P. D. Fagan, P. Mitrano, P. Sermanet, P. Abbeel, P. Sundaresan, Q. Chen, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Mart'in-Mart'in, R. Bajjal, R. Scalise, R. Hendrix, R. Lin, R. Qian, R. Zhang, R. Mendonca, R. Shah, R. Hoque, R. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Lin, S. Moore, S. Bahl, S. Dass, S. Sonawani, S. Tulsiani, S. Song, S. Xu, S. Haldar, S. Karamcheti, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Ramamoorthy, S. Dasari, S. Belkhale, S. Park, S. Nair, S. Mirchandani, T. Osa, T. Gupta, T. Harada, T. Matsushima, T. Xiao, T. Kollar, T. Yu, T. Ding, T. Davchev, T. Z. Zhao, T. Armstrong, T. Darrell, T. Chung, V. Jain, V. Kumar, V. Vanhoucke, V. Guizilini, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Chen, X. Wang, X. Zhu, X. Geng, X. Liu, X. Liangwei, X. Li, Y. Pang, Y. Lu, Y. J. Ma, Y. Kim, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Wu, Y. Xu, Y. Wang, Y. Bisk, Y. Dou, Y. Cho, Y. Lee, Y. Cui, Y. Cao, Y.-H. Wu, Y. Tang, Y. Zhu, Y. Zhang, Y. Jiang, Y. Li, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Ma, Z. Xu, Z. J. Cui, Z. Zhang, Z. Fu, and Z. Lin. Open X-Embodiment: Robotic learning datasets and RT-X models. <https://arxiv.org/abs/2310.08864>, 2023.
- [29] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh. Video generation models as world simulators. OpenAI Technical Report, 2024. URL <https://openai.com/index/video-generation-models-as-world-simulators/>.
- [30] Z. Yang, J. Teng, W. Zheng, M. Ding, S. Huang, J. Xu, Y. Yang, W. Hong, X. Zhang, G. Feng, D. Yin, Y. Zhang, W. Wang, Y. Cheng, B. Xu, X. Gu, Y. Dong, and J. Tang. CogVideoX: Text-to-video diffusion models with an expert transformer. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=LQzN6TRFg9>.
- [31] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. URL <https://arxiv.org/abs/2503.20314>.
- [32] B. Chen, D. Martí Monsó, Y. Du, M. Simchowitz, R. Tedrake, and V. Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/2aee1c4159e48407d68fe16ae8e6e49e-Abstract-Conference.html.
- [33] K. Song, B. Chen, M. Simchowitz, Y. Du, R. Tedrake, and V. Sitzmann. History-guided video diffusion. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/song25b.html>.
- [34] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, Y. Aytar, S. M. E. Bechtler, F. Behbahani, S. C. Y. Chan, N. Heess, L. Gonzalez, S. Osindero, S. Ozair, S. Reed, J. Zhang, K. Zolna, J. Clune, N. de Freitas, S. Singh, and T. Rocktäschel. Genie: Generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://proceedings.mlr.press/v235/bruce24a.html>.
- [35] Y. Tian, S. Yang, J. Zeng, P. Wang, D. Lin, H. Dong, and J. Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. *arXiv preprint arXiv:2412.15109*, 2024.
- [36] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. mimic-video: Video-action models for generalizable robot control beyond vlas. *arXiv preprint 2512.15692*, 2025.
- [37] S. L. Li, A. Zhang, B. Chen, H. Matusik, C. Liu, D. Rus, and V. Sitzmann. Controlling diverse robots by inferring jacobian fields with deep networks. *Nature*, 643:89–95, 2025. doi:10.1038/s41586-025-09170-0. URL <https://www.nature.com/articles/s41586-025-09170-0>.

- [38] Z. Teed and J. Deng. Raft: Recurrent all-pairs field transforms for optical flow, 2020. URL <https://arxiv.org/abs/2003.12039>.
- [39] C. Doersch, Y. Yang, M. Vecerik, D. Gokay, A. Gupta, Y. Aytar, J. Carreira, and A. Zisserman. Tapir: Tracking any point with per-frame initialization and temporal refinement, 2023. URL <https://arxiv.org/abs/2306.08637>.
- [40] A. W. Harley, Y. You, X. Sun, Y. Zheng, N. Raghuraman, Y. Gu, S. Liang, W.-H. Chu, A. Dave, P. Tokmakov, S. You, R. Ambrus, K. Fragkiadaki, and L. J. Guibas. AllTracker: Efficient dense point tracking at high resolution. In *ICCV*, 2025.
- [41] D. Zhang, F. Wang, M. Pollefeys, and H. Xu. Megaflow: Zero-shot large displacement optical flow. *arXiv preprint arXiv:2603.25739*, 2026.
- [42] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, V. Guizilini, D. A. Herrera, M. Heo, K. Hsu, J. Hu, M. Z. Irshad, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O’Neill, R. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. E. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn. Droid: A large-scale in-the-wild robot manipulation dataset. 2024.
- [43] T. T. Zhang, D. Pfrommer, C. Pan, N. Matni, and M. Simchowitz. Action chunking and exploratory data collection yield exponential improvements in behavior cloning for continuous control, 2025. URL <https://arxiv.org/abs/2507.09061>.
- [44] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [45] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.
- [46] H. J. T. Suh, T. Pang, T. Zhao, and R. Tedrake. Dexterous contact-rich manipulation via the contact trust region. *The International Journal of Robotics Research*, 2026. doi: [10.1177/02783649251398875](https://doi.org/10.1177/02783649251398875). URL <https://journals.sagepub.com/doi/10.1177/02783649251398875>.
- [47] T. Chen, J. Xu, and P. Agrawal. A system for general in-hand object re-orientation. In *Conference on robot learning*, pages 297–307. PMLR, 2022.
- [48] R. Ranftl, A. Bochkovskiy, and V. Koltun. Vision transformers for dense prediction, 2021. URL <https://arxiv.org/abs/2103.13413>.
- [49] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski. Dinov2: Learning robust visual features without supervision, 2024. URL <https://arxiv.org/abs/2304.07193>.

A Implementation Details

This appendix collects implementation details omitted from the main exposition: video-model architecture and training recipe (§A.1), video-model ablations (§A.2), inference constants (§A.3), datasets (§A.4), and tokenization and conditioning encoders (§A.5). We use the notation from Sections 2–3: N context frames, M predicted frames, and a committed prefix of length K .

A.1 Architecture and Training Recipe of Video Models

Backbone family. We instantiate the video planner F_θ with the Wan family of open-weight video diffusion transformers [31]. The video planner is always trained and used in observation space: it predicts future visual observations from a short observation history and, when available, a task specification. We do not attach an action head to the video model, and the video model training objective never uses robot actions. This design keeps the planner compatible with action-free video pretraining and separates visual planning from embodiment-specific control.

Model variants. We use two model scales throughout the paper. The Wan-1.4B model is used for development runs and video-model ablations, while the Wan-14B model is used for the main large-scale experiments. The 14B runs are initialized from pretrained Wan-family checkpoints; in the main robot-control setting, we use the Large Video Planner (LVP) initialization [17] when specified. For ablations, we also consider smaller models trained from different initializations to measure the effect of video pretraining. Table 3 summarizes the stable configuration details.

Table 3: **Video model variants.** We report only stable configuration details. The smaller model is used for development and ablations, while the 14B model is used for the main experiments.

Variant	Role in this paper	Initialization	Training hardware
Wan-1.3B	Development and ablations	Wan-family checkpoint	1 H100 GPU
Wan-14B	Main experiments	Wan-family / LVP checkpoint	1 H200 node, 8 GPUs

Robot video post-training. For robot-control experiments, we post-train the planner on robot video data using the same generative prediction objective as in video pretraining. The target remains future RGB observations rather than actions. This post-training adapts the model to robot scenes, camera viewpoints, objects, and contact dynamics, while preserving the action-free interface of the planner. Dataset-specific details, including camera views, resolutions, frame counts, and train/validation splits, are provided in Appendix A.4.

Video-to-video training format. Each training example consists of a context segment and a future segment. Given N_{ctx} context latent frames, where $N_{\text{ctx}} \leq N$, the model predicts a future look-ahead of M latent frames. The context frames are used as conditioning, and the training loss is applied only to the future frames. We use this video-to-video format because it matches closed-loop deployment: at test time, the robot provides a recent observation history, and the planner proposes a short visual future to be translated into actions.

Diffusion-forcing objective. We train the planner with diffusion forcing [32], following the history-guided / DFoT-style formulation [33]. In diffusion forcing, different frames in a sequence can be assigned different noise levels, allowing the model to condition on clean or partially noised history frames while denoising future frames. For a future latent frame z_i , we use the standard linear-flow diffusion target and optimize mean-squared error on the future segment:

$$\mathcal{L}_{\text{vid}}(\theta) = \mathbb{E} \left[\sum_{i=1}^M \|\hat{\mathbf{v}}_{\theta,i} - \mathbf{v}_i\|_2^2 \right], \quad (7)$$

where $\mathbf{v}_i$ is the diffusion target for the i -th future latent frame and $\hat{\mathbf{v}}_{\theta,i}$ is the model prediction. Context frames are used for conditioning and are not included in this loss.

Context and look-ahead. The maximum context length N and the future look-ahead length M are treated as design choices. Longer context can improve temporal consistency and help disambiguate contact state, while longer look-ahead gives the inverse dynamics module a richer visual path to

follow. However, longer sequences also increase memory and computation and can make generation harder. We therefore use short local look-aheads in the closed-loop controller and replan frequently from new observations. The ablations over context length, look-ahead length, and related planner choices are reported in Appendix A.2.

Multi-view formatting. When multiple camera views are available, we concatenate them in pixel space and treat the result as a single video frame. This simple formatting lets the same backbone consume third-person and wrist-camera observations without architectural changes. For cross-embodiment training, all embodiments are converted to a shared view layout; when an embodiment provides fewer views, the missing view slots are padded with blank frames. This convention lets a single video planner be trained on a mixture of embodiments while leaving embodiment-specific action grounding to the inverse dynamics module.

Training hardware and implementation scope. The Wan-14B planners are trained on one node of 8 H200 GPUs. The Wan-1.4B planners are trained on a single H100 GPU. All runs use mixed-precision training when supported by the hardware and software stack. We intentionally omit low-level optimizer settings, sharding policies, and batch-size details from this section because these are implementation-dependent and may vary across development runs.

A.2 Video Model Ablations

This section summarizes the video-planner ablations used for model selection and design validation. All reported numbers are validation MSE losses on held-out future-frame prediction, computed under the diffusion-forcing objective described in Appendix A.1. These losses are useful for comparing video-model configurations, but they are not a substitute for the closed-loop success metrics reported in the main experiments. To avoid over-interpreting development diagnostics, we report only stable comparisons and round all validation losses to two decimal places.

Initialization and model scale. Table 4 compares the effect of video-model initialization and scale. The randomly initialized model is trained only on robot videos, while the pretrained variants start from Wan-family or LVP checkpoints [17, 31]. Rows marked with † terminated before their planned training budget and should be interpreted as development diagnostics rather than fully controlled scaling results.

Table 4: **Video-model initialization and scale ablations.** Validation MSE loss is measured on held-out future latent prediction and rounded to two decimal places.

Name	Backbone	Init.	Dataset / views	N	M	Val. MSE ↓
Random init	Wan-14B	random	DROID triview	8	6	0.40
Wan-14B	Wan-14B	Wan checkpoint [31]	DROID triview	8	6	0.13
LVP-14B	Wan-14B	LVP warm-start [17]	DROID triview	8	6	0.10

The comparison supports two qualitative conclusions. First, pretrained video initialization substantially improves robot-video prediction: the Wan-initialized 14B model achieves much lower validation loss than the randomly initialized model under the same data and view configuration. Second, the LVP-initialized 14B model gives the lowest validation loss among the configurations we report, consistent with the intuition that stronger video priors improve visual planning.

Context and look-ahead. We treat the context length N and visual look-ahead length M as deployment hyperparameters rather than reporting them as a standalone video-quality table. Longer context can provide more temporal information and help disambiguate contact state, while longer look-ahead gives the inverse dynamics module a richer local visual path to follow. At the same time, longer sequences increase memory and computation and can make future prediction harder. For the main closed-loop controller, we therefore use short local look-aheads and rely on frequent replanning from new observations; the canonical inference quantities are summarized in Appendix A.3.

Camera-view configuration. Table 5 compares the camera views supplied to the DROID video planner. The triview configuration concatenates two external views and a wrist view; the reduced-view configurations remove one or more of these streams. Because the raw loss is computed on different

image widths, we normalize losses to the triview pixel budget by multiplying the measured validation loss by $576/W$, where W is the concatenated input width.

Table 5: **Camera-view ablations.** Validation MSE loss is measured on held-out DROID clips with the same view configuration used for training, normalized to the triview 128×576 pixel budget, and rounded to two decimal places.

Name	Views	Resolution	N	M	Norm. Val. MSE ↓
Triview	ext1 + ext2 + wrist	128×576	8	6	0.13
External + wrist	ext + wrist	128×384	8	6	0.21
Two external views	ext1 + ext2	128×384	8	6	0.11
Single external view	ext1	128×192	8	6	0.21
Wrist-only	wrist	128×192	8	6	0.61

The view ablation suggests that global scene context is especially important for video prediction. Using two external views gives the lowest normalized validation loss, likely because the model observes the object, robot, and workspace from stable third-person viewpoints. The triview model is slightly worse by this metric but remains attractive for policy use because it combines global context with the wrist view, which can provide useful local information near contact. Removing one external view degrades prediction quality, and the wrist-only model performs substantially worse, consistent with the difficulty of predicting the full scene from a narrow, moving camera. We therefore use multi-view inputs in the main real-robot planner whenever available.

A.3 Inference Constants and Closed-Loop Rollout

Video sampling. At each policy update, the planner encodes the most recent N observations and samples M future latent frames with the UniPC sampler used by the Wan backbone [31]. The context latents are kept fixed and only the future latents are denoised. Unless otherwise stated, we use 40 UniPC denoising steps per call.

Closed-loop execution. VERA runs as a receding-horizon controller. At each step, the planner predicts M future frames; the controller decodes the first K and hands them to the Jacobian IDM, which produces K actions by applying Eq. 6 to each adjacent pair (anchored at the current observation). After executing the chunk, the robot receives new observations and the planner is queried again. Frames beyond the committed prefix are discarded.

Canonical operating point. Table 6 lists the values used in the main experiments. We keep K small so the robot replans often from fresh observations — important for contact-rich manipulation.

Table 6: **Canonical closed-loop inference constants.**

Quantity	Symbol	Value
Context length (latent frames)	N	6
Predicted look-ahead (latent frames)	M	4
Committed prefix (latent transitions)	K	1
Video sampler steps	—	40 UniPC

The number of low-level robot commands per committed latent transition depends on the task control rate and the video frame stride, and is set per environment.

A.4 Datasets and Embodiments

We evaluate VERA across simulated and real-world manipulation settings spanning planar pushing, 7-DoF arm manipulation, and 16-DoF dexterous in-hand manipulation. Table 7 summarizes the main dataset and embodiment statistics. Across all settings, the video planner is trained on RGB observations only. The Jacobian IDM additionally uses paired robot actions and visual motion estimates to learn embodiment-specific video-to-action translation.

PushT-Sim. PushT-Sim is a planar pushing benchmark in which a circular 2-DoF pusher moves a T-shaped block toward a target pose. We use the expert demonstration set with 206 episodes. The

Table 7: **Datasets and embodiments.** “Resolution” denotes the RGB input resolution after view concatenation or padding. Episode counts refer to the main trajectories used for video-model post-training and/or Jacobian IDM training in each setting. Real-world performance is evaluated by executing the learned policy on physical robots rather than by held-out trajectory prediction.

Environment	Type	Robot DoF	Episodes	Resolution	Cameras
PushT-Sim	Sim	2	206	128×384	one top-down view, padded to a shared canvas
Panda-Sim (MimicGen)	Sim	7	9,000	128×128 128×256	eye-in-hand, agent-view, dualview
Allegro-Sim	Sim	16	2,126 (+401 aux.)	128×384	triview
Panda-Real	Real	7	39,816	128×576	two external views + wrist view
Allegro-Real	Real	16	60–217	128×256	dualview

single top-down RGB view is resized to 128×128 and placed into a padded 128×384 canvas so that the video planner can share the same multi-view formatting convention used in other settings. The action is represented as a normalized 2D pusher-position delta between adjacent observations. For Jacobian IDM training, visual motion is estimated from RGB observations. Success follows the standard PushT target-overlap criterion, and evaluation rollouts are capped at 200 environment steps.

MimicGen (Panda-Sim). MimicGen (Panda-Sim) consists of simulated Franka Panda manipulation trajectories generated with the MimicGen pipeline [45]. We use four tasks: `square`, `coffee`, `stack`, and `stack_three`, with 3,000, 2,000, 2,000, and 2,000 episodes respectively. Each episode provides an external agent-view camera and an eye-in-hand camera. We use 128×128 resolution for individual views and 128×256 for the concatenated dual-view input. The action space is a 7-DoF end-effector command with gripper state. Success is determined by the original MimicGen task specification.

Allegro-Sim. Allegro-Sim is a simulated cube reorientation setting with a 16-DoF Allegro hand. The action is a 16D joint-position target tracked by a low-level controller. The main task is relative cube reorientation, where the hand rotates the cube from its initial pose toward a target relative orientation. We use 2,126 planner-generated reorientation episodes for the main task. We additionally use 401 auxiliary random-motion episodes to provide local motion coverage for Jacobian IDM training; these auxiliary trajectories are not goal-directed demonstrations of the reorientation task. Three calibrated RGB views are concatenated width-wise into a 128×384 input. For Jacobian IDM training, we use paired visual observations, actions, and visual motion estimates from the same simulated trajectories.

Allegro-Real. Allegro-Real consists of real-world trajectories collected on a horizontally mounted Allegro hand. We use a Phase 1 dataset of 60 episodes and a larger Phase 2 pretraining mix of up to 217 episodes. The tasks involve clockwise and counter-clockwise cube reorientation. Two calibrated RGB cameras are concatenated into a 128×256 dual-view input. Evaluation is performed by executing the policy on the physical robot under held-out trial conditions.

Panda-Real. Panda-Real is our real-world Franka-family tabletop manipulation setting. For video-planner post-training, we use the successful subset of DROID [42], consisting of 39,816 real-robot trajectories collected with Franka-family arms and parallel grippers. We use two external cameras and one wrist camera, concatenated into a 128×576 RGB input. The video planner is trained on RGB observations only. For the Panda Jacobian IDM, we use the corresponding action streams in the Panda control parameterization and visual motion estimated from RGB observations. Unless otherwise noted, Panda actions are represented as end-effector delta commands with gripper state. Evaluation is conducted on our physical Panda setup with held-out prompts, object configurations, and camera placements.

Train / validation / evaluation protocol. For MimicGen, we hold out 250 episodes uniformly across tasks for validation and use the remaining episodes for training. Closed-loop simulation evaluation uses fresh rollout initializations or task instances rather than simply replaying held-out trajectories. For real-world settings, evaluation is performed by executing the learned policy on the physical robot

Algorithm 1 VERA policy

Require: video world model F_θ , Jacobian IDM G_ϕ , tokenizer E_{tok} , detokenizer D_{tok} , conditioning signal c , context length N , look-ahead length M , commit length $K_{\text{keep}} \leq M$, token stride r

- 1: **while** episode not terminated **do**
- 2: Encode context tokens $z_{t-N+1:t} \leftarrow E_{\text{tok}}(o_{t-N+1:t})$.
- 3: Sample future tokens $\hat{z}_{t+1:t+M} \sim F_\theta(\cdot \mid z_{t-N+1:t}, c)$.
- 4: Decode the committed prefix $\hat{o}_{t+1:t+K_{\text{keep}}} \leftarrow D_{\text{tok}}(\hat{z}_{t+1:t+K_{\text{keep}}})$.
- 5: Form $\hat{\tau}_{t:t+K_{\text{keep}}}^{\text{vid}} = (o_t, \hat{o}_{t+1}, \dots, \hat{o}_{t+K_{\text{keep}}})$.
- 6: Recover an action chunk $\hat{\tau}_{t:t+rK_{\text{keep}}-1}^a \leftarrow G_\phi(\hat{\tau}_{t:t+K_{\text{keep}}}^{\text{vid}})$.
- 7: Execute $\hat{\tau}_{t:t+rK_{\text{keep}}-1}^a$, append the new observations, and replan.
- 8: **end while**

and reporting task success over held-out trials. Across all environments, closed-loop task success is the primary evaluation metric; validation losses are used only for model selection and diagnostics.

A.5 Tokenization and Conditioning Encoders

Video tokenizer and decoder. We use the pretrained video tokenizer and decoder from the Wan model family [31]. The tokenizer maps RGB video clips into latent video tokens, and the decoder maps generated latent tokens back into RGB frames. Both components are kept fixed during the video model finetuning. This keeps the video planner objective in the same latent video space as the pretrained backbone and avoids introducing any action-specific representations into the video model.

Temporal layout. The Wan tokenizer uses a causal temporal compression. We denote by N the number of context latent frames and by M the number of predicted future latent frames. For the clip lengths used in our experiments, N latent context frames correspond to $4N-3$ RGB context frames, while M predicted latent frames correspond to $4M$ future RGB frames. These conversions are used only to describe the closed-loop operating point; the controller itself operates by encoding recent RGB observations, sampling future latent frames, decoding the committed prefix, and translating that visual prefix into actions with the Jacobian IDM.

Task conditioning. When task-conditioning signals are available, such as language prompts in real-world manipulation, the video planner conditions on them during video prediction. When such signals are not available or not used, the same architecture is trained as a video-to-video predictor from observation history alone. This allows the planner to support both prompt-conditioned real-robot settings and task-structured simulation settings without changing the video-to-action interface.

Multi-view formatting. For multi-camera settings, camera views are concatenated in pixel space before tokenization. The tokenizer therefore receives a single wide RGB frame containing all available views. This convention is simple but useful: it allows the same video backbone and tokenizer to handle single-view, dual-view, and triview inputs without architectural changes. For cross-embodiment video training, missing view slots are padded with blank frames so that all embodiments can share a common input layout.

Separation from actions. Robot actions are not tokenized by the video model and are never predicted by the video planner. Actions are used only to train the embodiment-specific Jacobian IDM. At deployment, the planner produces future visual observations, the decoder converts the committed latent prefix into RGB frames, and the Jacobian IDM translates those frames into low-level robot actions. This maintains the central factorization of VERA: action-free visual planning in the video model and embodiment-specific control in the inverse dynamics module.

B Jacobian Training Details

B.1 Jacobian Inversion

For the inverse-problem term of the joint training loss in Eq. 5, we require inverting the predicted Jacobian. We use ridge-regularized inversion, which has the closed-form solution:

$$\tilde{\mathbf{J}}_{\theta}^{-1} = (\mathbf{J}_{\theta}\mathbf{J}_{\theta}^{\top} + \lambda\mathbf{I})^{-1}\mathbf{J}_{\theta}^{\top}. \quad (8)$$

B.2 Jacobian Architecture Details

We instantiate the inverse-dynamics Jacobian model G_{ϕ} as a *Jacobian field*: given an RGB observation, the model outputs a dense, per-pixel tensor that maps a low-level robot command $\delta\mathbf{u}$ to a 2D optical-flow field. Given K command channels and a 2D pixel grid of size $H \times W$, the predicted Jacobian field has shape

$$\mathbf{J}_{\theta}(o) \in \mathbb{R}^{K \times 2 \times H \times W},$$

and the predicted flow under command $\delta\mathbf{u} \in \mathbb{R}^K$ is

$$\hat{f}(o, \delta\mathbf{u})_{:,h,w} = \mathbf{J}_{\theta}(o)_{:,:,h,w} \delta\mathbf{u},$$

i.e. a per-pixel matrix–vector product. We supervise this prediction against optical flow extracted from adjacent video frames during training (§A.1, §A.4), and at deployment we invert it to obtain $\delta\mathbf{u}$ from a generated visual flow (§B.1).

Implementations. We use two different architectures. At the small, simulation scale, we utilize a DPT neck and head [48] on top of a frozen DINOv2 encoder [49]. At the larger, real-world scale, we utilize and instantiate from the recent multi-view scale-up of this line of architectures, VGGT [44].

B.3 Jacobian Ablation Details

For ablations, we utilize the DINOv2 Jacobian model architecture. For complete fairness, for the D-IDMbaselines we utilize the same size DINOv2 encoder, same size DPT neck, and exchange out the decoding head for one which predicts in the action dimension rather than a spatial Jacobian. This results in matched parameter counts.



Figure 7: **Additional results on the hidden button task.** (Top) We visualize the dream rollout of our approach. (Bottom) We visualize the execution results of our model, finding that our model is capable of solving the task by finding the hidden button and pushing it.

C Additional Results and Qualitative Analyses

Fine-tuning is critical for robot planning. A central question is whether generic video pretraining transfers to robot planning. We compare two initialization regimes while keeping the robot data fixed: (i) training the planner from scratch on robot data only, and (ii) fine-tuning a pretrained Wan-family checkpoint on the same robot data. In our runs, pretrained initialization is substantially more data-efficient. Models trained from scratch do not reach the same level of temporal coherence or rollout stability within our compute budget, whereas fine-tuned pretrained models inherit useful

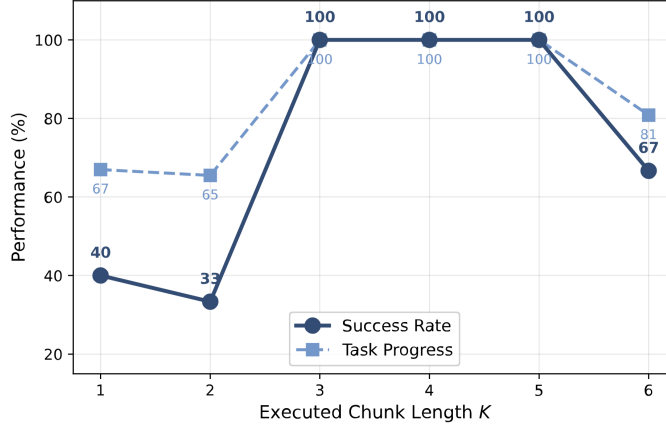


Figure 8: **Effect of action chunking on closed-loop performance.** We ablate the number of low-level actions executed before replanning. Increasing the chunk length initially improves performance by producing smoother local execution and reducing excessive replanning. However, overly long chunks reduce feedback frequency and can hurt performance, especially when contact or visual tracking errors accumulate. We therefore use an intermediate chunk length in the main experiments.

visual priors such as object permanence, lighting consistency, and contact geometry. For this reason, all main-result planners are initialized from pretrained video checkpoints, and from-scratch training is treated only as an ablation of pretraining.

Action chunking is important for closed-loop performance. In this section we ablate the planner hyperparameters that impact action chunking: the maximum context length N and the number of future latent frames M generated per planner call. We sweep $(N, M) \in \{(2, 2), (4, 4), (6, 4), (6, 8)\}$ and use $(N=6, M=4)$ as the default operating point. This setting provides enough history for stable rollouts while keeping future prediction tractable. Short context hurts long-horizon coherence, while aggressive look-ahead provides only limited benefit once frequent closed-loop replanning is used. We also found fixed-length future generation more reliable than variable-length generation, which otherwise introduces a mismatch between the training objective and the planner call pattern used at deployment.

On simulation pretraining and domain transfer. Pretraining components of the system on simulation data can improve downstream real-world performance when the simulation covers relevant local motion patterns and contact events. In our experiments, simulation data is most useful as a source of embodiment-specific motion supervision for the Jacobian IDM and as an additional source of robot-video regularization for the planner. At the same time, successful real-world transfer still depends on real-image adaptation, especially for appearance, lighting, and camera-geometry mismatch. We therefore view simulation pretraining as complementary to—rather than a replacement for—real-world post-training.